

PCFA 受講管理システム — 零基础面试手敲全指南

- > **作者目的**：魏玉臻用 — 不只是为了 6/29 发表，更要**面试时能讲清楚、能手敲一版**
- > **语言**：中文为主，必须懂的日文术语保留原文
- > **项目路径**：`lecture-management-system/demo`

目录

- [这个项目到底是什么?](#1-这个项目到底是什么)
- [技术栈一张表（面试必背）](#2-技术栈一张表面试必背)
- [AWS 架构（发表 + 面试必画）](#3-aws-架构发表--面试必画)
- [项目文件夹结构（每一块干什么）](#4-项目文件夹结构每一块干什么)
- [程序四层架构（手敲的核心逻辑）](#5-程序四层架构手敲的核心逻辑)
- [数据库表说明（Entity = 表）](#6-数据库表说明entity--表)
- [三种角色与全部画面 URL](#7-三种角色与全部画面-url)
- [登录流程（从零手敲第一步）](#8-登录流程从零手敲第一步)
- [S3 文件上传（课题 + 头像，面试高频）](#9-s3-文件上传课题--头像面试高频)
- [プロフィール功能（你新加的重点）](#10-プロフィール功能你新加的重点)
- [聊天功能（现状 + Lambda 说明）](#11-聊天功能现状--lambda-说明)
- [各功能模块代码地图](#12-各功能模块代码地图)
- [UI 主题 pcfa-theme.css](#13-ui-主题-pcfa-themecss)
- [部署流程（EC2 + Docker）](#14-部署流程ec2--docker)
- [面试模拟问答 20 题](#15-面试模拟问答-20-题)
- [手敲复现路线图（4 周计划）](#16-手敲复现路线图4-周计划)

1. 这个项目到底是什么？

日文名：PCFA 受講管理システム（PCFA 听课管理系统）

一句话：培训学校用的 Web 系统，管理员、讲师、学生三种人登录，管理课程、出席、教材、课题、聊天。

三种角色（ロール）：

日文		英文	中文	测试账号
管理者		ADMIN	管理员	admin@test.com
講師		INSTRUCTOR	讲师	teacher@test.com
受講者		STUDENT	学生	student@test.com

密码（测试环境）：`Admin1234`

2. 技术栈一张表（面试必背）

层次	技术	干什么用	面试怎么说
后端框架	Spring Boot 4 + Java 21	写服务器逻辑	「Spring MVC + JPA で REST ではなく SSR 構成です」
画面模板	Thymeleaf（タイムリーフ）	HTML 里嵌 Java 数据	「Controller から Model にデータを渡し、Thymeleaf で描画」
数据库	PostgreSQL on RDS	存用户、课程、出席等	「RDS PostgreSQL、JPA で ORM」
文件存储	AWS S3	PDF 教材、作业、头像	「Presigned URL でブラウザ直传」
服务器	EC2 + Docker + Nginx	跑 Java 程序	「EC2 上 Docker、Nginx がリバースプロキシ」
安全	BCrypt + Session	密码加密、登录状态	「セッションに loginUser を保存、ロールで画面分岐」
前端样式	Bootstrap 5 + 自定义 CSS	界面好看	「Bootstrap + 共通 CSS で統一」

没有用的东西（面试别乱说）：

- ✗ 没有 Cognito（没有邮箱验证）
- ✗ 没有 Lambda 实时聊天（聊天直接存 RDS）
- ✗ 没有前后端分离（不是 React/Vue，是传统 SSR）

3. AWS 架构 (发表 + 面试必画)

``

[浏览器 Browser]

| HTTP :80

▼

[EC2] — Nginx — Docker — Spring Boot (Java)

| |

| |— JDBC → [RDS PostgreSQL]

| |

| |— AWS SDK → [S3 桶]

| lecture-system-files-wei

|— 安全组 Security Group

- 80: 0.0.0.0/0 (谁都能看网站)

- 22: 特定 IP (只有允许的 IP 能 SSH)

、

Presigned URL 流程 (必考):

、

浏览器问服务器: 「我要上传文件」

服务器用 IAM 角色签名, 生成临时 URL (10分钟有效)

浏览器拿这个 URL 直接 PUT 文件到 S3 (不经过 EC2 内存)

服务器只在数据库记 s3Key (文件路径)

下载/显示时, 再生成 Presigned GET URL

、

为什么用 Presigned URL?

大文件不压 EC2

安全 (临时 URL 会过期)

4. 项目文件夹结构（每一块干什么）

、

demo/

- ├─ pom.xml # Maven 依赖清单（像 shopping list）
- ├─ Dockerfile # 怎么把 Java 打成 Docker 镜像
- ├─ docker-compose.yml # EC2 上一键启动 app + nginx + db
- ├─ src/main/
 - | └─ java/lecture_management_system/
 - | | └─ DemoApplication.java # 程序入口 main()
 - | | └─ config/
 - | | | └─ SecurityConfig.java # Spring Security（本项目几乎放行，靠 Session）
 - | | └─ controller/ # ★ 接待员：接收浏览器请求
 - | | | └─ LoginController.java
 - | | | └─ StudentController.java
 - | | | └─ InstructorController.java
 - | | | └─ AdminController.java
 - | | | └─ ChatController.java
 - | | └─ service/ # ★ 业务员：业务逻辑
 - | | | └─ UserService.java
 - | | | └─ CourseService.java
 - | | | └─ S3Service.java
 - | | | └─ ProfileService.java # 【新】头像プロフィール
 - | | | └─ ...

```

| | └─ repository/ # ★ 仓库：数据库 CRUD
| | | └─ UserRepository.java
| | | └─ ...
| | └─ entity/ # ★ 表结构定义（1 个类 = 1 张表）
| | | └─ User.java
| | | └─ Course.java
| | | └─ ...
| | └─ dto/ # 数据传输小对象
| └─ resources/
| └─ application.yaml # 配置（RDS 地址、S3 桶名）
| └─ static/
| | └─ pcfa-theme.css # 【新】全站绿色主题
| └─ templates/ # ★ HTML 页面（Thymeleaf）
| └─ login.html
| └─ student-home.html
| └─ student-profile.html # 【新】
| └─ fragments/
| | └─ profile-card.html # 【新】共用卡片
| └─ ...
└─ docs/ # 学习文档
、

```

记忆口诀：

> Controller 接请求 → Service 算逻辑 → Repository 查数据库 → Entity 是表 → templates 是画面

5. 程序四层架构（手敲的核心逻辑）

5.1 一次点击的完整旅程

以「学生点プロフィール編集」为例：

、

- ① 浏览器 GET /student/profile
- ② Spring 找到 StudentController.profilePage()
- ③ Controller 从 Session 取 loginUser
- ④ 调用 ProfileService.getAvatarUrl(user)
- ⑤ ProfileService 调用 S3Service.generatePresignedUrl()
- ⑥ Controller 把 avatarUrl 放进 Model
- ⑦ 返回字符串 "student-profile" → Thymeleaf 渲染 student-profile.html
- ⑧ HTML 送回浏览器

、

5.2 各层职责（面试背这个）

层	日文/英文		职责	不能干什么
Controller	コントローラ	接 HTTP、检查登录、调 Service、选模板		不写 SQL
Service	サービス	业务规则、组合多个 Repository		不直接碰 HTTP
Repository	リポジトリ	数据库增删改查		不写业务判断
Entity	エンティティ	表字段定义		不写逻辑

5.3 Session 登录（本项目没用 JWT）

```
`java

// 登录成功时（LoginController）

session.setAttribute("loginUser", user);
```

// 每个页面开头（所有 Controller）

```
User loginUser = (User) session.getAttribute("loginUser");

if (loginUser == null) return "redirect:/login";

、
```

面试问：为什么用 Session 不用 JWT？

答：社内研修システム、SSR 構成なのでセッションで十分。JWT は SPA 向け。

6. 数据库表说明（Entity = 表）

表名		Entity 类	存什么
users	User.java	用户（姓名、邮箱、密码、角色、头像路径、电话、自我介绍）	
courses		Course.java	课程名、说明
course_users	CourseUser.java	谁属于哪个班（学生/讲师和课程的多对多）	
course_schedules		CourseSchedule.java	上课日期
attendances		Attendance.java	出席记录
materials	Material.java	教材 PDF（s3Key）	
assignments		Assignment.java	课题（作业题目）
submissions		Submission.java	学生提交的作业 + 分数
chat_messages		ChatMessage.java	聊天消息

users 表新增字段（プロフィール功能）

列名	Java 字段	说明
avatar_s3_key	avatarS3Key	S3 上头像路径
phone	phone	电话
profile_bio	profileBio	自我介绍

所属コース怎么查？ 不存在 users 表里！通过 course_users 关联查：

```
`java

courseService.getStudentCourses(userId) // 返回 List<Course>

、
```

7. 三种角色与全部画面 URL

7.1 共通

URL	画面
GET /login	ログイン画面
POST /login	登录处理
GET /logout	ログアウト

7.2 受講者（学生）STUDENT

URL	画面	Controller
/student/home	ホーム	StudentController
/student/attendance?courseId=	出席登録	StudentController
/student/materials?courseId=	教材一覧	StudentController
/student/assignments?courseId=	課題提出	StudentController
/student/profile	プロフィール編集	StudentController
/student/profile/view	プロフィール预览	StudentController
/chat?courseId=	チャット	ChatController

7.3 講師（讲师）INSTRUCTOR

URL	画面
-----	----

/instructor/home	講師ホーム
/instructor/materials?courseId=	教材管理
/instructor/assignments?courseId=	課題管理
/instructor/submissions?courseId=	提出確認・採点
/instructor/attendance?courseId=	出欠確認
/instructor/students?courseId=	【新】受講者一覧
/instructor/students/{id}/profile?courseId=	学生プロフィール

7.4 管理者 ADMIN

URL	画面
/admin/home	管理者ポータル
/admin/accounts	ユーザー管理
/admin/accounts/{id}/profile	ユーザープロフィール
/admin/courses	コース管理
/admin/attendance	出欠修正
/admin/reports	実績レポート
/admin/chat	全チャット閲覧

8. 登录流程（从零手敲第一步）

8.1 手敲顺序

- User.java (entity)
- UserRepository.java (interface extends JpaRepository)
- UserService.java (findByEmail, checkPassword)
- LoginController.java
- login.html

8.2 LoginController 核心代码逻辑

、

POST /login 收到 email + password

→ `userRepository.findByEmail(email)`

→ 没有 → 错误消息

→ `locked == true` → 锁定消息

→ BCrypt 比对密码

→ 失败 → 失败次数+1，5次锁定

→ 成功 → `session.setAttribute("loginUser", user)`

→ 按 role 跳转 /admin/home 或 /instructor/home 或 /student/home

、

8.3 假邮箱问题

问：admin@test.com 能用吗？

答：能。系统只检查数据库有没有这个邮箱 + 密码对不对，不发验证邮件。

9. S3 文件上传（课题 + 头像，面试高频）

9.1 两种上传方式

方式		用于	流程
服务器中转	讲师上传教材 PDF	浏览器 → Spring MultipartFile → S3Service.uploadFile()	
Presigned URL 直传	学生交作业、上传头像	浏览器 → 拿 URL → 直接 PUT S3	

9.2 课题提出（学生）流程

、

GET /student/assignments/presign?fileName=xxx.pdf

返回 { uploadUrl, s3Key }

```
浏览器 fetch(uploadUrl, { method:'PUT', body: file })

POST /student/assignments/submit { assignmentId, s3Key, fileName }

Submission 表插入一条记录

、
```

9.3 S3Service 关键方法

方法	作用
uploadFile()	服务器直接上传
generatePresignedUrl()	生成下载/显示 URL（GET，10分钟）
generatePresignedImageUploadUrl()	生成上传 URL（PUT，头像用）
deleteFile()	删旧头像

EC2 不需要写 Access Key: IAM Role 自动认证。

10. プロフィール功能（你新加的重点）

10.1 新增/修改的文件清单

文件	作用
User.java	+avatarS3Key, phone, profileBio
ProfileService.java	头像 presign、保存、取 URL
AvatarPresignResult.java	小 DTO
S3Service.java	+generatePresignedImageUploadUrl
StudentController.java	/student/profile 系列
InstructorController.java	讲师看 profile + 受講者一覽
AdminController.java	管理员看 profile + courseList
student-profile.html	编辑页（含 JS 上传）

student-profile-view.html	预览
fragments/profile-card.html	共用详情卡片（含所属コース）
instructor-students.html	【新】讲师学生列表
pcfa-theme.css	头像圆圈样式

10.2 头像上传 JS 逻辑（student-profile.html）

```
`javascript`

// 简化版理解

选文件 → GET /student/profile/presign-avatar?fileName=xx.jpg

→ 拿到 uploadUrl 和 s3Key

→ PUT uploadUrl（文件直接到 S3）

→ 用户点保存 → POST /student/profile（s3Key 写入数据库）

、
```

10.3 谁能看谁

角色	入口
学生	自己编辑 /student/profile
讲师	ホーム→受講者一覧→プロフィール
管理者	ユーザー管理→プロフィール

10.4 所属コース显示原理

```
`java`

// AdminController.userProfile() 里

model.addAttribute("courseList", courseService.getStudentCourses(user.getId()));

// profile-card.html 里

<span th:each="c : ${courseList}" th:text="${c.name}"></span>
```

11. 聊天功能（现状 + Lambda 说明）

11.1 现在怎么实现的

、

学生/讲师 POST /chat/send → ChatService.sendMessage()

→ chat_messages 表 INSERT

→ 页面 redirect 刷新显示

管理者 GET /admin/chat → 看全部消息

管理者 POST /admin/chat/delete/{id} → 删除

、

没有实时推送：发完要刷新页面才看到新消息。

11.2 Lambda 为什么「感觉没用」

你可能之前计划：消息 → Lambda → 通知

但现在代码没用 **Lambda**，消息直接存 RDS。Lambda 要另外接 API Gateway + SNS 才有「新消息通知」效果。

11.3 发表后改进方案（面试可说）

阶段		方案	难度
简单	chat_messages 加 is_read，ホーム显示未读数		★★
中等	前端每 30 秒 AJAX 轮询		★★★
复杂	WebSocket 或 Lambda + SNS 推送		★★★★★

发表时说法：

＞ チャットは RDS 保存の基本機能を実装済み。未読通知は今後の改善として既読フラグを追加予定です。

12. 各功能模块代码地图

出席 (**Attendance**)

`StudentController.registerAttendance()`

`AttendanceService` – 检查今天是不是上课日、有没有重复签到

`student-attendance.html`

教材 (**Material**)

`InstructorController` – 讲师上传 PDF

`MaterialService` + `S3Service`

`StudentController` – 学生看列表、Presigned 下载

课题 (**Assignment**) & 提出 (**Submission**)

`InstructorController` – 创建课题

`StudentController` – Presigned 上传 + 提交

`InstructorController` – 採点 (分数 0-100 + 评论)

`instructor-submissions.html`

実績レポート (**Report**)

`AdminController` + `ReportService`

按课程显示: 出席率、提出率、平均分数

`admin-reports.html`

コース管理

`AdminController` – 创建课程、加讲师学生、设上课日

`course_users` 表维护班级成员

13. UI 主题 `pcfa-theme.css`

作用: 全站统一绿色风格, 不用每个 HTML 各写一遍。

用法 (每个 HTML 加两行):

```
`html
```

```
<link rel="stylesheet" th:href="@{/pcfa-theme.css}">
```

```
<body class="pcfa-body">
```

、

主要 CSS 类:

类名	用途
pcfa-navbar	绿色顶栏
pcfa-course-card	课程卡片
pcfa-course-action	均等按钮
pcfa-avatar-sm / lg	头像圆圈
pcfa-profile-card	プロフィール详情卡片
pcfa-course-badge	所属コース绿色标签

背景色变量: `--pcfa-bg: #eef2f6` (浅灰, 不是纯白)

14. 部署流程 (EC2 + Docker)

14.1 本地 → GitHub

```
\bash

cd demo

git add .

git commit -m "说明写了什么"

git push

、
```

14.2 EC2 更新

```
\bash
```

```
cd ~/lecture-management-system
```

```
git pull
```

```
./mvnw clean package -DskipTests
```

```
sudo docker restart app-app-1
```

、

14.3 常见问题

问题	原因	解决
git pull 没变化	没 push	本地先 push
网页没变化	浏览器缓存	Ctrl+Shift+R
SSH 连不上	安全组 22 只允许旧 IP	加「マイ IP」规则
本地 compile 失败	Java 17, 项目要 21	在 EC2 上编译

15. 面试模拟问答 20 题

Q1. 这个项目你负责了什么？

A: 受講管理システムの学生向け UI 改善、S3 プロフィール機能、講師の受講者一覧、AWS デプロイ。

Q2. 为什么用 Spring Boot？

A: Java 企业标准，自动配置，整合 JPA/Security/Thymeleaf 快。

Q3. Thymeleaf 是什么？

A: 服务器端模板。Controller 传 Model, HTML 里 th:text 显示数据。

Q4. JPA 是什么？

A: 用 Java 类操作数据库，不用手写 SQL。Entity 对应表，Repository 对应 CRUD。

Q5. Presigned URL 是什么？

A: AWS 签发的临时 URL，让浏览器直连 S3 上传下载，减轻服务器压力。

Q6. 密码怎么存？

A: BCrypt 哈希, 不可逆。登录时 BCrypt.matches() 比对。

Q7. 三种角色怎么区分?

A: users.role 字段, 登录后跳不同 home, Controller 里检查 role。

Q8. 学生和课程什么关系?

A: course_users 中间表, 多对多。

Q9. 头像存在哪?

A: 图片在 S3, 路径在 users.avatar_s3_key。

Q10. 为什么 Session 不用 JWT?

A: 传统 SSR, 服务器渲染, Session 更简单。

Q11. ddl-auto: update 什么意思?

A: 启动时自动根据 Entity 更新表结构 (开发用, 生产要小心)。

Q12. Docker 干什么?

A: 打包运行环境, EC2 上 docker restart 重启应用。

Q13. Nginx 干什么?

A: 反向代理, 80 端口转发到 Spring Boot 8080。

Q14. 聊天怎么存的?

A: chat_messages 表, 没用 Lambda。

Q15. 怎么防止重复签到?

A: AttendanceService 查今天是否已有记录。

Q16. 课题截止怎么判断?

A: assignments.deadline 和学生提交时间比较。

Q17. 実績レポート算什么?

A: ReportService 聚合出席率、提出率、平均分。

Q18. 如果文件超过 1MB 上传失败?

A: application.yaml 配 multipart max-file-size: 10MB。

Q19. 你遇到的最大困难？

A: （诚实说）AWS 部署、安全组 SSH、Java 版本、Git push 流程。

Q20. 下一步改进？

A: チャット未読通知、UI 继续打磨、Cognito 认证。

16. 手敲复现路线图（4 周计划）

第 1 周：骨架

☐ Spring Initializr 创建项目（Web, JPA, PostgreSQL, Thymeleaf）

☐ User + Login + Session

☐ login.html + 三种 home 空页面

第 2 周：核心业务

☐ Course + CourseUser

☐ 出席、教材（先不用 S3，本地文件）

☐ 课题 + 提交

第 3 周：AWS

☐ S3 Presigned URL

☐ RDS 连接

☐ EC2 Docker 部署

第 4 周：进阶

☐ プロフィール + 头像

☐ チャット

☐ 実績レポート

☐ UI 主题 CSS

每天手敲 1 小时比背 10 小时更有效。

附录 A: Thymeleaf 常见写法

```
`html`

<!-- 显示变量 -->

<span th:text="${user.name}"></span>

<!-- 条件 -->

<div th:if="${avatarUrl}">有头像</div>

<div th:unless="${avatarUrl}">没头像</div>

<!-- 循环 -->

<tr th:each="course : ${courseList}">

<td th:text="${course.name}"></td>

</tr>

<!-- 链接 -->

<a th:href="@{/student/home}">ホーム</a>

<a th:href="@{/chat(courseId=${course.id})}">チャット</a>

<!-- 引入片段 -->

<div th:replace="~{fragments/profile-card :: card(${user}, ${avatarUrl}, ${courseList})}">

</div>

、
```

附录 B: 关键注解（看到要认识）

注解		意思
@Controller	这是接收网页请求的控制器	
@GetMapping("/path")		处理 GET 请求
@PostMapping("/path")		处理 POST 请求

@Autowired	Spring 自动注入依赖
@Entity	这是数据库表
@Service	这是业务服务类
@RequestParam	取 URL 参数 ?courseId=1
@PathVariable	取路径 /users/{id} 里的 id

附录 C：你新增的 commit 摘要

commit: feat: student profile S3 avatar + full UI theme`（及后续讲师一覧）

主要内容：

- 全站 pcfa-theme.css 绿色 UI
- S3 头像 Presigned 上传
- プロフィール编辑/预览/管理者讲师查看
- profile-card 显示所属コース
- 讲师ホーム→受講者一覧→プロフィール

文档版本：2026-06-23 | 配合项目 commit a366b78 及之后改动